

# ITS4243 - MICROSERVICES AND CLOUD COMPUTING

**Faculty of Technology - University of Sri Jayewardenepura**

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science

# T2. MICROSERVICES DESIGN PRINCIPLES

# What is a Microservice?

Microservices are an architectural approach to developing software applications as a collection of small, independent services that communicate with each other over a network.

Each service focuses on a single business capability.

Instead of building a monolithic application where all the functionality is tightly integrated into a single codebase, microservices break down the application into smaller, loosely coupled services. Can be written in a variety of programming languages, and frameworks, and each service acts as a mini-application on its own.

# Why choose microservices?

- Independent deployment & faster release cycles
- Technology heterogeneity (polyglot stacks)
- Scalability per service (right-size resources)
- Fault isolation
- Organizational alignment: teams own services

# Core Design Principles

- Single Responsibility and Bounded Context
- Loose Coupling
- High Cohesion
- Service Autonomy
- API-first Design
- Database per Service
- Asynchronous Communication & Event-Driven Patterns
- Observability (logging, metrics, tracing)
- Resiliency & Fault Tolerance
- Automation (CI/CD)
- Security by Design

# ❖ Concept 1: Single Responsibility Principle (SRP)

## Definition:

A microservice should have **one clear business responsibility** or **purpose** — it should *do one thing well*.

## Explanation:

- Derived from the SRP in software engineering (each class/module should have one reason to change).
- For microservices, this means each service changes only when its specific business function changes.
- Simplifies maintenance, testing, and deployment — no ripple effects across unrelated domains.

## Example:

In a university management system:

- Student-Service → manages student profiles, enrollment info
- Course-Service → manages course details, schedules
- Payment-Service → handles tuition payments

→ □ If course fees change, only Payment-Service updates — not the entire system.

## Benefits:

- Reduces complexity per service
- Makes ownership clear for teams
- Enables independent scaling and deployment
- Improves resilience (failure isolation)

## Common Mistake:

- Splitting too small — “nano-services” cause communication overhead.  
Example: separate services for *student-name* and *student-address* is excessive.

## ❖ Concept 2: Bounded Context (from Domain-Driven Design)

### Definition:

A *bounded context* defines the **logical boundary of a domain model** — where a particular business meaning and set of rules apply.

### In microservices:

Each microservice aligns with **one bounded context**. Inside that boundary, terms, entities, and operations have clear meaning and consistency.

### Example:

In an e-commerce system:

- “Order” means a *purchase request* inside the **Order Context**
  - In the **Inventory Context**, “Order” might mean a *replenishment request* to suppliers
- □ Same term, different meaning — so they must belong to separate bounded contexts (services).

## Benefits:

- Avoids data and terminology conflicts
- Keeps models consistent within each service
- Enables teams to work independently with their own domain understanding

## Visualization (you can show on the slide):

```
+-----+           +-----+
|  Order Context    |           | Payment Context    |
|-----|           |-----|
|  Order Entity     |           | Payment Entity     |
| OrderService API  |<----->| PaymentService API  |
+-----+           +-----+

(different models, same term "Order" used safely)
```

- How SRP and Bounded Context work together

| Concept               | Focus                   | Benefit                     |
|-----------------------|-------------------------|-----------------------------|
| Single Responsibility | One purpose per service | Simplicity, independence    |
| Bounded Context       | Clear business boundary | Consistency, domain clarity |

Together they ensure:

- **Cohesive functionality** within each service
- **Minimal coupling** across services
- **Business-aligned architecture**, not just technical modularity

# Loose Coupling & High Cohesion

## Concept 1: Loose Coupling

- **Definition:**  
Microservices should **interact with each other through well-defined, minimal dependencies** — one service should know *only what it needs* to know about another.
- **Explanation:**
- Changes in one service should **not force changes in others**.
- Services communicate via **stable interfaces (APIs, events)**, not through shared databases or internal classes.
- The goal: each service can evolve independently.

## **Technical Practices for Loose Coupling:**

### **1.API Contracts**

- Define and share using OpenAPI (Swagger).
- Keep backward compatibility (e.g., add fields instead of removing).

### **2.Consumer-Driven Contracts (CDC)**

- Example: using PACT testing to ensure service updates don't break consumers.

### **3.Event-Driven Communication**

- Services publish events instead of making direct synchronous calls when possible.
- e.g., OrderCreated → published, InventoryService and PaymentService listen.

### **4.Avoid Shared Databases or Shared Models**

- Each service owns its own schema.
- No direct queries into another service's database.

### **5.Versioned APIs**

- /v1/orders → /v2/orders instead of breaking existing consumers.

### **6.Independent Deployability**

- You should be able to deploy one service without redeploying others.

## Concept 2: High Cohesion

- **Definition:**

Cohesion measures **how closely related the functionalities within a service are.**

A highly cohesive service has *all components working together toward one well-defined purpose.*

- **Explanation:**

- All classes, modules, and data in a service should serve the same **business goal.**
- Avoid “miscellaneous” or unrelated responsibilities inside a single service.
- Improves maintainability, readability, and domain clarity.

## **Example:**

### **✓ High Cohesion:**

OrderService - handles only order creation, update, status tracking, and retrieval.

### **✗ Low Cohesion:**

OrderService - also manages payments, notifications, and reporting — these should be separate services.

### **Benefits of High Cohesion:**

- Easier to understand, test, and modify
- Fewer side effects when changes are made
- Encourages domain-aligned code organization

Think of a **car**:

- Each component (engine, transmission, brakes) performs one clear role (high cohesion).
- Components connect through standardized interfaces (loose coupling).
  - □ You can replace the engine without changing the brake system — exactly how microservices should behave.

## ◆ Relationship Between the Two

| Concept        | Goal                              | How Achieved                              | Benefit                           |  |
|----------------|-----------------------------------|-------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------|
| Loose Coupling | Reduce inter-service dependencies | API contracts, async events, no shared DB | Independent deployment            |                                                                                     |
| High Cohesion  | Keep related logic together       | DDD, clear service purpose                | Easier maintenance, focused teams |                                                                                     |

**Together** → Autonomy, Scalability, Maintainability

Tightly Coupled (Bad)

```
+-----+ +-----+
| OrderService |<-->| Payment DB |
|              | | Shared Table |
+-----+ +-----+
```

Loosely Coupled (Good)

```
+-----+ +-----+
| OrderService |--->| PaymentService |
| REST / Event | | API / Queue   |
+-----+ +-----+
```

# Best Practices Summary

- ✓ Design services around **business capabilities**, not technical layers (avoid “user-controller-service” split across systems).
- ✓ Define **clear interfaces** and avoid leaking internal logic.
- ✓ Communicate asynchronously where possible (to decouple timing).
- ✓ Ensure each team can **own and evolve** their service without waiting for others.

# Service Autonomy

- **Definition:**

A microservice is **autonomous** when it can be **developed, deployed, scaled, and operated independently** — without depending on other services' runtime, deployment, or data.

- **In simple terms:**

Each service is a *self-contained unit of work* that can **run on its own, fail without breaking others, and evolve at its own pace.**

## ◆ Key Dimensions of Autonomy

| Aspect              | Description                                                                    | Example                                                                                           |
|---------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Deployment Autonomy | A service can be built, tested, and deployed without coordinating with others. | <code>User-Service</code> can deploy new version without redeploying <code>Order-Service</code> . |
| Runtime Autonomy    | The service can operate even when other services are down (within reason).     | <code>Notification-Service</code> queues messages if <code>Email-Service</code> is unavailable.   |
| Data Autonomy       | Each service manages its own database and schema.                              | <code>Payment-Service</code> stores its own transactions, not in <code>Order</code> DB.           |
| Team Autonomy       | Each microservice is owned by a small, cross-functional team.                  | "You build it, you run it." (Amazon's 2-pizza team rule)                                          |

# Why Service Autonomy Matters

## ✓ **Independent Development:**

Teams can work on different services simultaneously.

## ✓ **Independent Deployment:**

No “big bang” releases; deploy small updates anytime.

## ✓ **Fault Isolation:**

If one service fails, others continue running.

## ✓ **Independent Scalability:**

Scale only the bottleneck service — cost-efficient and performant.

## ✓ **Faster Innovation:**

Teams can choose their own tech stack or version upgrades.

# Practical Example

## E-commerce system:

- Order-Service places orders
- Payment-Service processes payments
- Notification-Service sends confirmations

If the **Notification-Service** crashes, orders and payments still succeed; notifications are retried later.  
That's autonomy in action — services interact, but don't depend on each other's *availability* or *internal data*.

# Techniques to Achieve Autonomy

- **1. Own Data (Database per Service)**
  - Each service owns its data store.
  - No shared database or direct queries across services.
  - Use APIs or events to share necessary data.
- **2. Asynchronous Communication**
  - Use message queues (Kafka, RabbitMQ) or event streams.
  - Decouples service lifecycles (publisher doesn't wait for consumer).
- **3. API Contracts**
  - Stable, versioned APIs allow independent evolution.
  - Use OpenAPI or gRPC definitions as the contract.
- **4. Infrastructure Isolation**
  - Containerize each service (Docker)
  - Deploy on independent pods (Kubernetes)
  - Separate resource scaling (CPU, memory, autoscaling rules)
- **5. Team Ownership & DevOps**
  - The same team that builds the service also operates it (monitoring, alerting, incident management).
  - Enables accountability and rapid iteration.
- **6. Graceful Degradation**
  - Services continue operating with limited functionality if dependencies fail (use fallbacks or cached data).



# API-First Design

- API-first means you **design, document, and agree upon the service's API contract** *before* starting development — treating the API as the **primary product** of your service.
- **Simple idea:**
  - ☐ *The API is the interface between services and teams — it must come first.*
- **Goal:**

To make integration predictable, minimize rework, and ensure consistency across teams.

## ◆ Why API-First?

| Benefit                     | Description                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------|
| Consistency                 | All teams follow standard API conventions (naming, versioning, errors).             |
| Parallel Development        | Frontend, backend, and other microservices can develop in parallel using mock APIs. |
| Clear Contracts             | Reduces misunderstandings — both sides know expected inputs, outputs, and behavior. |
| Easier Testing & Automation | Auto-generate mocks, test clients, and SDKs from OpenAPI specs.                     |
| Governance & Reuse          | Ensures organizational standards and reusability across services.                   |

# The API-First Workflow

## **Design**

- Define resources, methods, and data models using OpenAPI/Swagger or AsyncAPI (for events).
- Example: YAML/JSON spec written collaboratively.

## **Review & Approve**

- Teams review the contract before coding begins (like a design doc).
- Use mock servers to test early.

## **Implement**

- Developers code the service according to the contract.
- Clients can already build against mock APIs.

## **Validate**

- Use tools like Swagger UI, Postman, or Insomnia for validation.
- Contract tests (CDC) ensure compatibility.

- This document becomes the **single source of truth** for developers, QA, and clients.

Example: OpenAPI Contract (snippet)

```
yaml
openapi: 3.0.3
info:
  title: Order Service API
  version: 1.0.0
paths:
  /orders:
    post:
      summary: Create a new order
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/OrderRequest'
      responses:
        '201':
          description: Order created
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrderResponse'
components:
  schemas:
    OrderRequest:
      type: object
      properties:
        customerId: { type: string }
        items:
          type: array
          items: { type: string }
    OrderResponse:
      type: object
      properties:
        orderId: { type: string }
        status: { type: string }
```

## ◆ Tools Commonly Used

- **Design & Documentation:** SwaggerHub, Stoplight Studio, Postman, Insomnia
- **Mocking & Testing:** Prism, WireMock, PACT (for consumer-driven contracts)
- **Validation & Linting:** Spectral (enforce API standards)
- **Code Generation:** OpenAPI Generator (creates server stubs & SDKs in multiple languages)

## ◆ API Versioning Best Practices

- **Always version your APIs** — avoid breaking existing consumers.

Examples:

- **URI Versioning** → `/api/v1/orders`
- **Header Versioning** → `Accept: application/vnd.myapp.v1+json`
- **Deprecate gracefully:** document old versions and provide migration timelines.

## ◆ API Governance (for large teams)

- Define **organization-wide API standards** for naming, status codes, error structures, and pagination.
- Use **API linting** and review gates before deployment.
- Maintain a **central API catalog** for discovery and reuse.

- Visual Slide Idea
- [ Step 1: Design Contract ] --> [ Step 2: Mock & Review ] --> [ Step 3: Implement ] --> [ Step 4: Validate & Deploy ]

Underneath:

“Don’t code first and document later — design together and code confidently.”

# Database per Service

- **Definition:**

Each microservice must **own and manage its own database**.

No two microservices should **directly share the same database schema** — this enforces **loose coupling** and **data autonomy**.

- **Why This Principle Matters**

- In a monolithic system:

- All modules share a **single centralized database**.

- Changing one table can unintentionally break another part of the system.

- In microservices:

- Each service manages its **own data and persistence layer**, ensuring full independence in evolution, scaling, and deployment.

Each service **owns** its schema and **only exposes** necessary data via an **API** or **event**.

Example (E-commerce System):

| Microservice    | Database    | Description                                |
|-----------------|-------------|--------------------------------------------|
| Order-Service   | orders_db   | Stores orders, order status, order history |
| Product-Service | products_db | Manages product catalog, stock details     |
| Payment-Service | payments_db | Handles transactions, receipts, refunds    |

# ◆ Benefits

## ✓ **Loose coupling:**

Services don't depend on others' database structures or queries.

## ✓ **Independent scaling:**

A high-traffic service (e.g., Orders) can scale its DB without affecting others.

## ✓ **Independent deployment:**

Schema changes in one service don't require coordination with other teams.

## ✓ **Technology freedom:**

Each service can choose the most suitable database — SQL, NoSQL, or graph — based on its domain.

Example:

- **Product-Service** → MongoDB (document-based for product catalogs)
- **Order-Service** → PostgreSQL (transactional)
- **Analytics-Service** → Elasticsearch (search and aggregation)

## ✓ **Improved data security and ownership:**

A service defines exactly what other services can access.

## ◆ Challenges & Solutions

| Challenge             | Solution                                                                                           |
|-----------------------|----------------------------------------------------------------------------------------------------|
| Data duplication      | Accept <i>some duplication</i> — data consistency is managed through APIs or event-driven updates. |
| Cross-service queries | Use <i>API composition</i> or <i>CQRS patterns</i> instead of direct joins.                        |
| Data consistency      | Apply <i>eventual consistency</i> using <i>domain events</i> or message brokers (Kafka, RabbitMQ). |
| Complex reporting     | Use a separate <i>data warehouse</i> or <i>data lake</i> to combine data periodically.             |

```
Order-Service (orders_db)
|
|--> Event: "OrderCreated"
v
Inventory-Service (inventory_db)
```

## .Design Pattern Example — *Event-Driven Data Sharing*

Instead of direct DB access:

- 1.Order-Service creates an order
- 2.It **publishes an event**: OrderCreated
- 3.Inventory-Service subscribes to the event and updates its stock
- 4.No direct database coupling — only asynchronous event communication.

**Visualization:**